ANKARA ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM 4522 PROJE RAPORU

Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Özge TARAŞLI

21290755

Github Adı: ozgetarasli

Github Proje Linki:

https://github.com/ozgetarasli/ag-tabanli-paralel-dagitim-sistemleri-projeleri/tree/main/Veritabani_Yedekleme_ve_Kurtarma

Video Linki:

https://ankarauniv-my.sharepoint.com/:v:/g/personal/21290755_ogrenci_ankara_edu_tr/IQCMMXBsrD1hR6WObK_hO1-CAWv0fL_AzW2xTNSe1hhjluw?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=cDp8HN

ÖZET

Bu rapor, Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında gerçekleştirilen “Veritabanı Yedekleme ve Felaketten Kurtarma Planı” projesini açıklamak amacıyla hazırlanmıştır. Çalışmada, Microsoft SQL Server üzerinde Northwind örnek veritabanı kullanılarak tam yedekleme (full backup), fark yedekleme (differential backup) ve işlem günlüğü yedekleme (transaction log backup) yöntemleri uygulanmıştır.

Proje kapsamında yalnızca yedek alma işlemleri değil, aynı zamanda olası veri kaybı durumlarında uygulanacak felaketten kurtarma adımları da ele alınmıştır. Oluşturulan senaryoda, veritabanında test amaçlı bir tablo hazırlanmış, tabloya kritik veriler eklenmiş ve ardından bu verilerden bir kısmı silinerek veri kaybı durumu simüle edilmiştir. Sonrasında tam yedek, işlem günlüğü yedeği ve point-in-time restore yaklaşımı kullanılarak silinen verilerin geri getirilmesi sağlanmıştır.

Bunun yanında, alınan yedeklerin güvenilirliğini doğrulamak amacıyla Restore Verifyonly komutu ve side-by-side restore yöntemi kullanılmıştır. Böylece hem yedekleme zincirinin doğru çalıştığı hem de alınan yedeklerin kullanılabilir olduğu gösterilmiştir.

Sonuç olarak bu proje, veritabanı sistemlerinde veri kaybını önlemek, iş sürekliliğini korumak ve olası felaket senaryolarında hızlı geri dönüş sağlamak için etkili bir yedekleme ve kurtarma planının nasıl oluşturulabileceğini uygulamalı olarak ortaya koymaktadır.

İÇİNDEKİLER

ÖZET 2
İÇİNDEKİLER 3
1. GİRİŞ 4
   1.1 Kullanılan Ortam 4
   1.2 Veritabanı Kurulumu 4
   1.3 Amaç ve Planlama 5
2. YEDEKLEME STRATEJİLERİ 6
   2.1 Tam Yedekleme(Full Backup) 6
   2.2 Fark Yedekleme(Differential Backup) 6
   2.3 İşlem Günlüğü Yedekleme(Transaction Log Backup) 7
3. ZAMANLAYICILARLA OTOMATİK YEDEKLEME 8
4. FELAKETTEN KURTARMA SENARYOSU 10
   4.1 Test Tablosu Oluşturma 10
   4.2 Veri Eklenmesi 10
   4.3 Backup Alınması 11
   4.4 Verilerin Silinmesi 11
   4.5 Tail-Log Backup Alınması 12
   4.6 Veritabanının Geri Yüklenmesi 12
5. YEDEKLERİN TEST EDİLMESİ 14
   5.1 Restore Verifonly 14
   5.2 Side By Side Restore 14
6. SONUÇ 15

1. GİRİŞ

Bu proje, veritabanlarında veri güvenliğini sağlamak amacıyla yedekleme ve felaketten kurtarma yöntemlerinin uygulanmasını kapsamaktadır. Çalışmada Microsoft SQL Server üzerinde Northwind veritabanı kullanılarak tam yedekleme (Full Backup), fark yedekleme (Differential Backup) ve işlem günlüğü yedekleme (Transaction Log Backup) yöntemleri uygulanmıştır.

Ayrıca, yedekleme işlemleri SQL Server Agent ile otomatik hale getirilmiş ve oluşturulan yedeklerin gerçek bir veri kaybı senaryosunda kullanılabilirliği test edilmiştir. Bu kapsamda, silinen veriler point-in-time restore yöntemi ile geri getirilmiştir.

Bu raporda, yedekleme stratejileri ve felaketten kurtarma süreci adım adım açıklanmıştır.

1.1 Kullanılan Ortam

Bu proje Microsoft SQL Server 2022 üzerinde gerçekleştirilmiştir. Veritabanı yönetimi için SQL Server Management Studio (SSMS) kullanılmıştır

1.2 Veritabanı Kurulumu

Proje kapsamında örnek veritabanı olarak Northwind kullanılmıştır. Northwind, müşteri, sipariş ve ürün bilgilerini içeren klasik bir örnek veritabanıdır. Veritabanı sisteme başarıyla yüklenmiş ve gerekli kontroller yapılarak aktif olduğu doğrulanmıştır ve aşağıdaki sorgu ile doğrulanıp veritabanını yüklediğimiz gösterilmiştir:

SQLQuery17...B\MSI (83))* | SQLQuery16.s...AB\MSI (65))*

```sql
SELECT
    d.name AS DatabaseName,
    d.state_desc AS Status,
    d.recovery_model_desc AS RecoveryModel
FROM sys.databases d
WHERE d.name = 'Northwind';
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

	DatabaseName	Status	RecoveryModel
1	Northwind	ONLINE	FULL

1.3 Amaç ve Planlama

Bu projenin amacı, olası veri kayıplarına karşı etkili bir yedekleme ve kurtarma planı oluşturmaktır. Bu doğrultuda aşağıdaki adımlar izlenmiştir.

Tam Yedekleme (Full Backup): Veritabanının tamamının o anki hali kopyalanarak ana yedek oluşturulması.

Fark Yedeklemesi (Differential Backup): Sadece tam yedekten sonra değişen veriler yedekleme.

Log Yedeklemesi (Transaction Log): Gün içindeki anlık tüm işlemler kayıt altına alınması.

Otomatikleştirme: Tüm bu yedekleme süreçlerinin SQL Agent aracılığıyla belirli bir takvime göre kendiliğinden çalışması sağlanması.

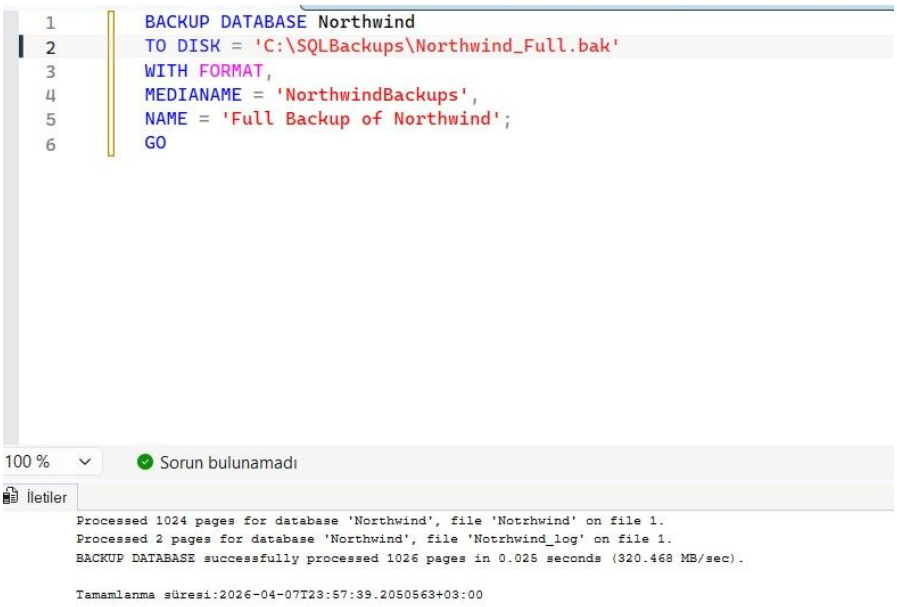
Test ve Kurtarma: Sistemin düzgün çalıştığından emin olmak için veriler bilerek bozulmuş veya silinmiştir. Ardından, Point-in-Time Restore yöntemiyle veritabanı tam olarak istenilen eski bir ana başarıyla geri döndürülmüştür.

2. YEDEKLEME STRATEJİLERİ

Bu bölümde, veritabanı yedekleme sürecinde kullanılan temel yöntemler ele alınmıştır. Yedekleme stratejileri, veri kaybını önlemek ve gerektiğinde veriyi belirli bir zamana geri döndürebilmek için kritik öneme sahiptir.

2.1 Tam Yedekleme (Full Backup)

Tam yedekleme, veritabanının tamamının tek bir dosya halinde yedeklenmesidir. Bu yöntem, diğer yedekleme türlerinin temelini oluşturur ve genellikle belirli aralıklarla alınır. Geri yükleme işlemlerinde ilk adım olarak kullanılır.



Şekil 1: Tam Yedekleme Ekran Görüntüsü

2.2 Fark Yedekleme (Differential Backup)

Fark yedekleme, son alınan tam yedekten sonra değişen verileri içerir. Tam yedeğe göre daha hızlı alınır ve daha az yer kaplar. Geri yükleme sırasında, tam yedek ile kullanılır.

Tam yedekleme her seferinde veritabanının tamamını kopyalarken, fark yedekleme yalnızca son tam yedekten bu yana değişen blokları kaydeder. Bu sayede hem yedekleme süresi hem de kullanılan depolama alanı önemli ölçüde azalır. Örneğin,

tam yedek haftalık alınırken fark yedekleme günlük olarak alınabilir; bu sayede olası bir veri kaybında yalnızca tam yedek ve en güncel fark yedeğinin geri yüklenmesi yeterli olur.

Fark yedekleme, özellikle veritabanının büyük ancak değişim oranının düşük olduğu ortamlarda tercih edilir. Transaction log yedeklemeye kıyasla daha az sıklıkta alınır; bu nedenle yüksek frekanslı geri yükleme ihtiyacı olan sistemlerde transaction log yedekleme ile kullanılması önerilir.

```
26  BACKUP DATABASE Northwind
27  TO DISK = 'C:\SQLBackups\Northwind_Diff.bak'
28  WITH DIFFERENTIAL,
29  NAME = 'Differential Backup of Northwind';
30  GO
31
```

```
%   Sorun bulunamadı
letiler
Processed 72 pages for database 'Northwind', file 'Notrhwind' on file 1.
Processed 2 pages for database 'Northwind', file 'Notrhwind_log' on file 1.
BACKUP DATABASE WITH DIFFERENTIAL successfully processed 74 pages in 0.017 seconds (33.777 MB/sec).

Tamamlanma süresi:2026-04-08T00:00:36.5825136+03:00
```

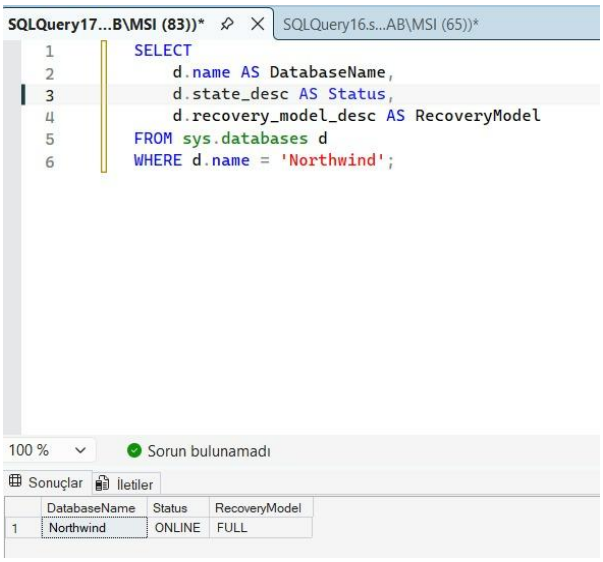
Şekil 2: Fark Yedekleme Ekran Görüntüsü

2.3 İşlem Günlüğü Yedekleme (Transaction Log Backup)

İşlem günlüğü yedekleme, veritabanında gerçekleşen tüm işlemleri kaydeder. Bu sayede veritabanı belirli bir zamana (point-in-time) geri döndürülebilir. Bu yöntem yalnızca Full Recovery Model kullanıldığında aktif olarak çalışır.

```
BACKUP LOG Northwind
TO DISK = 'C:\Backups\Northwind_Log.trn'
WITH NAME = 'Transaction Log Backup of Northwind';
GO
```

Şekil 3: Transaction Log Backup Ekran Görüntüsü



Şekil 4: Full Modelde Olduğumuzu Gösteren Ekran Görüntüsü

3. ZAMANLAYICILARLA OTOMATİK YEDEKLEME

Yedekleme stratejilerinin manuel yürütülmesi veri kaybı riskini barındırır. Bu çalışmada SQL Server Agent kullanılarak günlük tam yedeklerin otomatikleştirilmesi sağlanmıştır.

İlk olarak özel bir Görev (Job) oluşturulur.

```
EXEC dbo.sp_add_job
    @job_name = N'Northwind_Daily_Full_Backup',
    @enabled = 1,
    @description = N'Northwind veritabanı için günlük otomatik tam yedekleme görevi.';
GO
```

Bu Job'ın içine BACKUP DATABASE komutu adım/steps olarak eklenir.

```
EXEC sp_add_jobstep
    @job_name = N'Northwind_Daily_Full_Backup',
    @step_name = N'Execute Full Backup',
    @subsystem = N'TSQL',
    @command = N'BACKUP DATABASE Northwind TO DISK = ''C:\Backups\Northwind_Daily.bak'' WITH
    INIT',
    @retry_attempts = 3,
    @retry_interval = 5;
GO
```

Zamanlayıcı oluşturulur ve örneğin her gece saat 02:00 ayarlanarak oluşturulan Görev'e bağlanır

```sql
-- Zamanlayıcı oluşturma
EXEC sp_add_schedule
    @schedule_name = N'Daily_2AM_Schedule',
    @freq_type = 4,          -- Günlük (Daily)
    @freq_interval = 1,      -- Her gün (Every 1 day)
    @active_start_time = 020000; -- Saat 02:00:00
GO

-- Göreve bağlama

EXEC sp_attach_schedule
    @job_name = N'Northwind_Daily_Full_Backup',
    @schedule_name = N'Daily_2AM_Schedule';
GO
```

100 % Sorun bulunamadı

Sonuçlar İletiler

	JobAdi	CalismaTarihi	CalismaSaati	Durum	Mesaj
1	Northwind - Günlük Tam Yedekleme	20260407	213451	Succeeded	The job succeeded. The Job was invoked by User D...

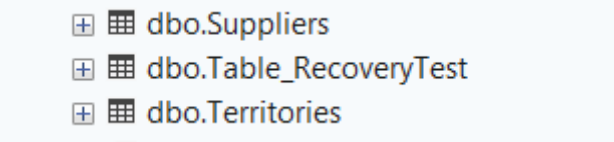
Şekil 5: Otomatik yedekleme oluştuğunun kanıtı

4. FELAKETTEN KURTARMA SENARYOSU

4.1 Test Tablosu Oluşturma

İlk olarak test tablosu oluşturuyoruz. Felaket kurtarma senaryosunu test edebilmek amacıyla `Table_RecoveryTest` adında bir test tablosu oluşturduk. Bu tablo, veri ekleme ve silme işlemlerini simüle ederek geri yükleme sürecinin doğruluğunu test etmek için kullanılacaktır.

```sql
CREATE TABLE Table_RecoveryTest (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Data NVARCHAR(100),
    CreatedAt DATETIME DEFAULT GETDATE()
);
```



Şekil 6: Tablonun Oluştuğunu Gösteren Ekran Görüntüsü

4.2 Veri Eklenmesi

Test senaryosu kapsamında ikinci olarak tablomuza verileri ekliyoruz. Bu veriler, felaket sonrası geri yükleme işleminin doğruluğunu test etmek amacıyla kullanılacaktır.

```sql
INSERT INTO Table_RecoveryTest (Data)
VALUES
('Onemli Musteri Verisi 1'),
('Onemli Siparis Verisi 2'),
```

Sonuçlar | İletiler

	Id	Data	CreatedAt
1	1	Onemli Musteri Verisi 1	2026-04-09 20:43:27.637
2	2	Onemli Siparis Verisi 2	2026-04-09 20:43:27.637

Şekil 7: Verilerin Eklendiğini Gösteren Ekran Görüntüsü

4.3 Backup Alınması

Verilerin güvenli bir noktası oluşturmak amacıyla veritabanının tam yedeği (full backup) alınmıştır. Bu yedek, felaket sonrası geri dönüş için temel referans noktası olarak kullanılacaktır.

```sql
BACKUP DATABASE Northwind
TO DISK = 'C:\SQLBackups\Northwind_Full.bak'
WITH INIT, NAME = 'Full Backup Before Disaster';
```

```
)0 %    Sorun bulunamadı
İletiler
Processed 1064 pages for database 'Northwind', file 'Notrhwind' on file 1.
Processed 2 pages for database 'Northwind', file 'Notrhwind_log' on file 1.
BACKUP DATABASE successfully processed 1066 pages in 0.015 seconds (554.947 MB/sec).

Tamamlanma süresi:2026-04-09T20:45:18.4447261+03:00
```

Verilerin henüz silinmediği anın zaman bilgisi kaydedilmiştir. Bu zaman bilgisi, point-in-time restore işlemi sırasında kullanılacaktır.

```sql
SELECT GETDATE() AS CurrentTime;
```

	CurrentTime
1	2026-04-09 20:48:34.983

Şekil 8: Zamanın Kaydedildiğini Gösteren Ekran Görüntüsü

4.4. Verilerin Silinmesi

Felaket senaryosu kapsamında, kritik verilerden biri yanlışlıkla silinmiş gibi simüle edilmiştir. Bu işlem, gerçek hayatta kullanıcı hatası sonucu oluşabilecek veri kaybını temsil etmektedir.

```sql
DELETE FROM Table_RecoveryTest
WHERE Data = 'Onemli Musteri Verisi 1';
```

Sonuçlar | İletiler

	Id	Data	CreatedAt
1	2	Onemli Siparis Verisi 2	2026-04-09 20:43:27.637

Şekil 9: Tablodan Veri Silindiğini Gösteren Ekran Görüntüsü

4.5. Tail-Log Backup Alınması

Veri kaybını minimuma indirmek amacıyla Tail-Log Backup alınmıştır. Bu işlem, veritabanının kapanmadan önceki son transaction kayıtlarını koruyarak geri yükleme zincirine dahil edilmesini sağlar. Böylece tam yedek ile veri silme işlemi arasındaki süreçteki tüm değişiklikler de kurtarma sürecine dahil edilmiş olur.

```sql
BACKUP LOG Northwind
TO DISK = 'C:\SQLBackups\Northwind_Tail_Log.trn'
WITH NORECOVERY;
```

0 % Sorun bulunamadı

İletiler

```
Processed 63 pages for database 'Northwind', file 'Notrhwind_log' on file 2.
BACKUP LOG successfully processed 63 pages in 0.008 seconds (61.035 MB/sec).

Tamamlanma süresi:2026-04-09T20:53:17.6737294+03:00
```

Şekil 10:Tail-Log Backup Alındığını Gösteren Ekran Görüntüsü

4.6. Veritabanın Geri Yüklenmesi

Veritabanı geri yükleme işlemi restore zinciri mantığına uygun şekilde gerçekleştirilmiştir. İlk olarak tam yedek geri yüklenmiş ve veritabanı NORECOVERY modunda bırakılmıştır. Ardından transaction log yedeği kullanılarak veritabanı, veri kaybı yaşanmadan önceki zamana geri getirilmiştir.

STOPAT parametresi kullanılarak veritabanı belirli bir zamana kadar geri alınmış ve RECOVERY ile aktif hale getirilmiştir.

```sql
RESTORE LOG Northwind
FROM DISK = 'C:\SQLBackups\Northwind_Tail_Log.trn'
WITH STOPAT = '2026-04-09 20:53:17',
RECOVERY;
```

Sonuçlar | İletiler

	ID	Data	InsertTime
1	1	Onemli Müsteri Verisi 1	2026-04-08 00:09:51.600
2	2	Onemli Siparis Verisi 2	2026-04-08 00:09:51.600

Şekil 11: Veritabanının Geri Yüklendiğini Gösteren Ekran Görüntüsü

Geri yükleme işlemi tamamlandıktan sonra yapılan kontrollerde silinen verinin başarıyla geri getirildiği gözlemlenmiştir. Bu durum, point-in-time restore işleminin doğru şekilde çalıştığını göstermektedir.

5. YEDEKLERİN TEST EDİLMESİ

Alınan yedeklerin güvenilirliği ve sağlamlığı kriz anından önce periyodik olarak test edilmelidir. Bunu da biz iki şekilde göstererek doğruladık.

5.1. Restore Verifonly:

Veritabanına fiziksel veya mantıksal bir zarar gelip gelmediğini, dosyaların sağlamlığını verileri geri yüklemeden test eder.

```
RESTORE VERIFYONLY
FROM DISK = 'C:\SQLBackups\Northwind_Full.bak';
GO
```

00 % Sorun bulunamadı

İletiler

```
The backup set on file 1 is valid.

Tamamlanma süresi:2026-04-08T00:33:19.7974641+03:00
```

5.2. Side By Side Restore:

Üretim ortamını tehlikeye atmadan yedeğin işlevselliğini %100 kanıtlamak adına yedek dosyasından (`.bak`) geçici bir test veritabanı kurulur ve çalıştırılır.

SQLQuery22.s...AB\MSI (55))* | SQLQuery21.s...AB\MSI (91))* | SQLQuery20.s...AB\MSI (92))* | SQ

```
USE master;
GO

RESTORE DATABASE Northwind_TestRestore
FROM DISK = 'C:\SQLBackups\Northwind_Full.bak'
WITH
    MOVE 'Notrhwind' TO 'C:\SQLBackups\Northwind_Test_Data.mdf',
    MOVE 'Notrhwind_log' TO 'C:\SQLBackups\Northwind_Test_Log.ldf',
    RECOVERY;
GO
```

100 % Sorun bulunamadı

İletiler

```
Processed 1024 pages for database 'Northwind_TestRestore', file 'Notrhwind' on file 1.
Processed 2 pages for database 'Northwind_TestRestore', file 'Notrhwind_log' on file 1.
RESTORE DATABASE successfully processed 1026 pages in 0.072 seconds (111.273 MB/sec).

Tamamlanma süresi:2026-04-08T00:37:47.3762588+03:00
```

6. SONUÇ

Bu projede, Microsoft SQL Server üzerinde Northwind veritabanı kullanılarak yedekleme ve felaketten kurtarma süreçleri uygulamalı olarak gerçekleştirilmiştir. Tam, fark ve işlem günlüğü yedekleme yöntemleri kullanılarak veri güvenliği sağlanmış ve bu süreçler otomatik hale getirilmiştir.

Oluşturulan felaket senaryosu ile veri kaybı durumu simüle edilmiş ve point-in-time restore yöntemi kullanılarak silinen veriler başarıyla geri getirilmiştir. Ayrıca, alınan yedeklerin doğruluğu test edilerek sistemin güvenilirliği doğrulanmıştır.

Sonuç olarak, doğru planlanmış bir yedekleme ve kurtarma stratejisinin veri kaybını önlemede ve sistem sürekliliğini sağlamada kritik bir rol oynadığı görülmüştür.